

Informe: “Presentación 2”

Asignatura: INF331 - Pruebas de software

Profesor: Oscar Rodrigo Reyes Hevia

Repositorio: <https://github.com/presentacion-2/presentacion-2>

Link video: <https://youtu.be/psBfnqfpTMw>

Integrantes:

Javier Norambuena

Bastián Cornejo

Katty Herrera



Tecnologías

Jenkins: Es una aplicación de código abierto especializada para la implementación de CI/CD. Por lo tanto, su objetivo es el poder automatizar procedimientos de integración de nuevas características y versiones. Jenkins fue desarrollada en java y fue lanzada su primera versión el año 2011.

Para lograr esta automatización, se utilizan pipelines, los cuales agrupan los procesos de build, test y deployment, entre otros. Jenkins trae la posibilidad de instalar diferentes plugins que permiten integrar diversas tecnologías que se ocupan en el desarrollo de software (Slack, Jenkins, Selenium entre otros).

Selenium: Es una popular herramienta de automatización de pruebas utilizada en el desarrollo de aplicaciones web. Es una biblioteca de código abierto que proporciona una interfaz fácil de usar para interactuar con diferentes navegadores web, como Chrome, Firefox y Safari. Con Selenium, los desarrolladores y testers pueden automatizar acciones repetitivas, como navegar por páginas web, interactuar con elementos, completar formularios y verificar resultados, lo que ayuda a mejorar la eficiencia y la calidad de las pruebas en entornos web.

Además de su flexibilidad y compatibilidad con múltiples lenguajes de programación, en esta ocasión decidimos utilizar Python Selenium debido a su amplia adopción y su sólida integración con el ecosistema de Python. Python es conocido por su sintaxis clara y legible, lo que facilita la escritura y el mantenimiento del código de automatización de pruebas. Además, Selenium proporciona una API completa y bien documentada en Python, lo que nos permite aprovechar todas sus funcionalidades para interactuar con aplicaciones web de manera eficiente y confiable.



Descripción del trabajo

Para esta presentación se tomará como base el trabajo realizado por el grupo 8 en la presentación 1, es decir, la aplicación de la biblioteca de la Universidad de Salamanca. La idea es tomar ese software y desarrollar un entorno de CI/CD para el manejo de versiones e integración de nuevos features en la rama de producción. La aplicación de la biblioteca, para este caso, será levantada con un contenedor docker con imagen **selenium/node-firefox**.

Para llevar a cabo la implementación de un sistema CI/CD se utilizará un pipeline soportado en Jenkins. Este pipeline estará compuesto por tres stages. El primero es el encargado de construir la imagen del contenedor y dejarlo ejecutando, la segunda stage se encarga de ejecutar las pruebas en selenium y la tercera stage es generar el despliegue en producción.

Debido a que se tiene Jenkins y la aplicación de forma local en un mismo computador, se decidió simular los entornos de producción y desarrollo con carpetas distintas, es decir, una carpeta tendrá el código de la última versión de la aplicación y otra carpeta es donde se hacen los nuevos cambios. La razón detrás de esto es porque no se tiene un entorno propio de producción.

Jenkins se levanta de forma local y se genera una url pública utilizando ngrok para crear el webhook de github, el pipeline se ejecuta solo cuando el push se realiza a la rama main del repositorio.

Para la sección de testing del pipeline, se utilizará la herramienta de selenium junto con el lenguaje de programación Python para ejecutar las pruebas unitarias. En un principio se tenía planeado ejecutar las pruebas utilizando Google Chrome, pero debido a incompatibilidades con la arquitectura arm64, se tuvo que cambiar por el navegador Mozilla Firefox y geckodriver. La decisión tomada por el equipo respecto a cómo construir las pruebas se basó en probar las distintas funciones que un usuario puede realizar en la plataforma, las cuales son descritas en la siguiente sección.



Conjunto de pruebas

En esta sección, se describe el conjunto de pruebas que se realizó utilizando la herramienta Selenium, con el propósito de evaluar el funcionamiento, identificar posibles errores y asegurar la calidad del software.

A continuación se mostrarán los detalles de los casos de pruebas definidos por el equipo:

ID	Descripción	Entrada	Resultado Esperado	Resultado de Ejecución
TC_0001	Hacer login como usuario alumno/profesor	User: test1@gmail.com Psw: 123	Interfaz de alumno/profesor	Pass
TC_0002	Hacer login como usuario administrador	User: admin@gmail.com Psw: 123	Interfaz de administrador	Pass
TC_0003	Buscar un libro existente como alumno/profesor	Nombre: Deep Learning	Se muestra libro en el buscador	Pass
TC_0004	Ver detalle de un libro existente	Nombre: Deep Learning	Se muestra detalle de libro	Pass
TC_0005	Crear un libro y revisar que el libro haya sido creado	Título: "The Hunger Games" Año: "2008" Autor: "Suzanne Collins" Editorial: "Scholastic Press" Descripción: "The Hunger Games is a 2008 dystopian novel by the American writer Suzanne Collins. It is written in the voice of 16-year-old Katniss Everdeen, who lives in the future, post-apocalyptic nation of Panem in North America."	Libro aparece al buscarlo	Pass
TC_0006	Editar título de un libro existente y revisar que estos cambios se hayan realizado	Título: Deep	Nombre de título modificado exitosamente	Pass
TC_0007	Eliminar un libro y	Título: "The	Libro no	Pass



	revisar que ya no exista	Hunger Games”	aparece al buscarlo	
TC_0008	Buscar un libro que no existe	Título: El principito	Resultado de búsqueda tabla vacía	Pass
TC_0009	Agregar libro con información incompleta	Título: “The Hunger Games” Autor: “Suzanne Collins” Editorial: “” Año: “2008” Descripción: “”	Libro se ingresa correctamente y aparece al buscarlo	Pass
TC_0010	Buscar libros por autor	Autor: “Thomas”	Se muestra 1 libro del autor	Pass
TC_0011	Buscar libros por editorial	Editorial: “MIT”	Se muestran 3 libros del autor	Pass
TC_0012	Realizar búsqueda vacía	Búsqueda vacía y sin filtro	Se muestran todos los libros	Pass
TC_0013	Hacer login con credenciales inválidas	User: “Test1@gmail.com” psw:”1234”	No se inicia sesión y muestra mensaje de error	Pass
TC_0014	Modificar el estado de un libro	Libro: Deep Learning Estado cambia de Disponible a Reservado	Cambia el estado del libro a Reservado	Pass



Pipeline

Como ya se dejó ver, el pipeline fue construido utilizando Jenkins, la base del pipeline es un proyecto libre, al cual se le conecta con un webhook de github mediante la url obtenida de ngrok. Dentro de este se generan dos stages para la ejecución de código.

La primera stage es la encargada de construir la imagen de docker de la aplicación de la biblioteca. Primero actualiza las ramas correspondientes para luego hacer una verificación de que no existan contenedores viejos aún activos, una vez hecho esto procede a construir la imagen en base al archivo dockerfile definido en el repositorio. Cuando la imagen está completa, se procede a levantar el contenedor.

git checkout main
git pull origin main

```
if [[ $(/usr/local/bin/docker ps -aqf "name=getting-started") ]]; then  
  /usr/local/bin/docker stop getting-started  
  /usr/local/bin/docker rm getting-started  
  /usr/local/bin/docker rmi getting-started:latest  
fi
```

```
/usr/local/bin/docker build -t getting-started .  
/usr/local/bin/docker run -d -p 2222:22 --name getting-started getting-started
```

La segunda stage corresponde a la ejecución de las pruebas en selenium, para ello, nos conectamos remotamente al contenedor levantado en la stage anterior y dentro de este ejecutamos el comando para ejecutar las pruebas usando pytest.

```
/usr/local/bin/docker exec --workdir /app/biblioteca-app/frontend/src/ __tests__  
getting-started pytest --html=report.html -k test_TC_0001.py  
/usr/local/bin/docker exec --workdir /app/biblioteca-app/frontend/src/ __tests__  
getting-started pytest --html=report.html -k test_TC_0002.py  
/usr/local/bin/docker exec --workdir /app/biblioteca-app/frontend/src/ __tests__  
getting-started pytest --html=report.html -k test_TC_0003.py  
/usr/local/bin/docker exec --workdir /app/biblioteca-app/frontend/src/ __tests__  
getting-started pytest --html=report.html -k test_TC_0004.py  
/usr/local/bin/docker exec --workdir /app/biblioteca-app/frontend/src/ __tests__  
getting-started pytest --html=report.html -k test_TC_0005.py  
/usr/local/bin/docker exec --workdir /app/biblioteca-app/frontend/src/ __tests__  
getting-started pytest --html=report.html -k test_TC_0006.py  
/usr/local/bin/docker exec --workdir /app/biblioteca-app/frontend/src/ __tests__  
getting-started pytest --html=report.html -k test_TC_0007.py  
/usr/local/bin/docker exec --workdir /app/biblioteca-app/frontend/src/ __tests__  
getting-started pytest --html=report.html -k test_TC_0008.py  
/usr/local/bin/docker exec --workdir /app/biblioteca-app/frontend/src/ __tests__  
getting-started pytest --html=report.html -k test_TC_0009.py  
/usr/local/bin/docker exec --workdir /app/biblioteca-app/frontend/src/ __tests__  
getting-started pytest --html=report.html -k test_TC_0010.py
```



```
/usr/local/bin/docker exec --workdir /app/biblioteca-app/frontend/src/__tests__  
getting-started pytest --html=report.html -k test_TC_0011.py  
/usr/local/bin/docker exec --workdir /app/biblioteca-app/frontend/src/__tests__  
getting-started pytest --html=report.html -k test_TC_0012.py  
/usr/local/bin/docker exec --workdir /app/biblioteca-app/frontend/src/__tests__  
getting-started pytest --html=report.html -k test_TC_0013.py  
/usr/local/bin/docker exec --workdir /app/biblioteca-app/frontend/src/__tests__  
getting-started pytest --html=report.html -k test_TC_0014.py
```

```
/usr/local/bin/docker stop getting-started  
/usr/local/bin/docker rm getting-started  
/usr/local/bin/docker rmi getting-started:latest
```

Después de ejecutar las pruebas y que estas no fallen, se procede a detener y eliminar el contenedor junto a la imagen para no ocupar espacio. El porque las pruebas se ejecutan de forma secuencial y no con un solo comando es temas de espacio, si se ejecutan con un solo comando el contenedor muere por falta de recursos.

La tercera stage corresponde al deploy de la nueva versión de la aplicación en la carpeta de producción, el código de abajo corresponde al script asociado al stage.

```
cd /Users/jnorambu/Documents/produccion  
git pull origin main  
echo "Actualizacion completada"
```

Debido a la configuración de las ramas, estas tres líneas solo se ejecutan en la rama main. Considerando lo expuesto anteriormente, lo primero es movernos al entorno de producción y aplicar el pull request para actualizar a la nueva versión del código.



Conclusiones

Después de trabajar con estas nuevas tecnologías, desde el lado de Jenkins se puede concluir que es una herramienta muy versátil, con una disponibilidad de varios tipos de plugins para integrar tecnologías, a pesar de que varias ya no tienen soporte.

Selenium es una herramienta altamente útil y superior a las herramientas de prueba anteriores cuando se trata de probar el DOM. Su flexibilidad se destaca al ofrecer múltiples versiones que se adaptan a diferentes lenguajes de programación según nuestras necesidades. Una ventaja adicional es que permite la observación de la ejecución de las pruebas, lo que brinda claridad a los testers al enfrentarse a errores, ya que facilita la identificación de soluciones y los pasos a seguir durante la prueba.

Ya hablando desde la integración nos dimos cuenta de la importancia del diseño no solo del pipeline si no que de la aplicación, si bien posible aun así crear el pipeline y ejecutar las pruebas, no tener en cuenta el cómo está construida la aplicación puede llevar a problemas en la implementación del pipeline y la ejecución de pruebas. En nuestro caso tuvimos que introducir la aplicación en un contenedor para poder ejecutar la aplicación sin tener que complejizar de forma innecesaria el script del stage.

A pesar de todo, Jenkins y Selenium fueron buenas herramientas para el desarrollo de un proceso CI/CD, si hubiéramos definido de mejor forma la arquitectura del proceso de CI/CD, la implementación hubiera sido más directa y sencilla.



Anexo de errores encontrados

En el desarrollo se encontró un problema que a la fecha de entrega no pudo ser solucionado. El problema ocurre en la etapa de ejecución de los test, es decir, cuando el contenedor con la aplicación está funcionando. Este problema corresponde a que al momento de comprobar ciertos elementos en la ventana del navegador estos no se encuentran haciendo que los test fallen. El porqué esto es un error, se debe a que cuando se ejecutan de forma local, estos elementos si se encuentran. Hipotetizamos que el error se debe a que el contenedor se está ejecutando en un Mac (arquitectura arm64), y jenkins y selenium tienen problemas en este tipo de máquinas, un ejemplo de esto es que el contenedor debe ser ejecutado con una arquitectura amd64 (se simula esta arquitectura en Mac) para que los test puedan ser ejecutados.

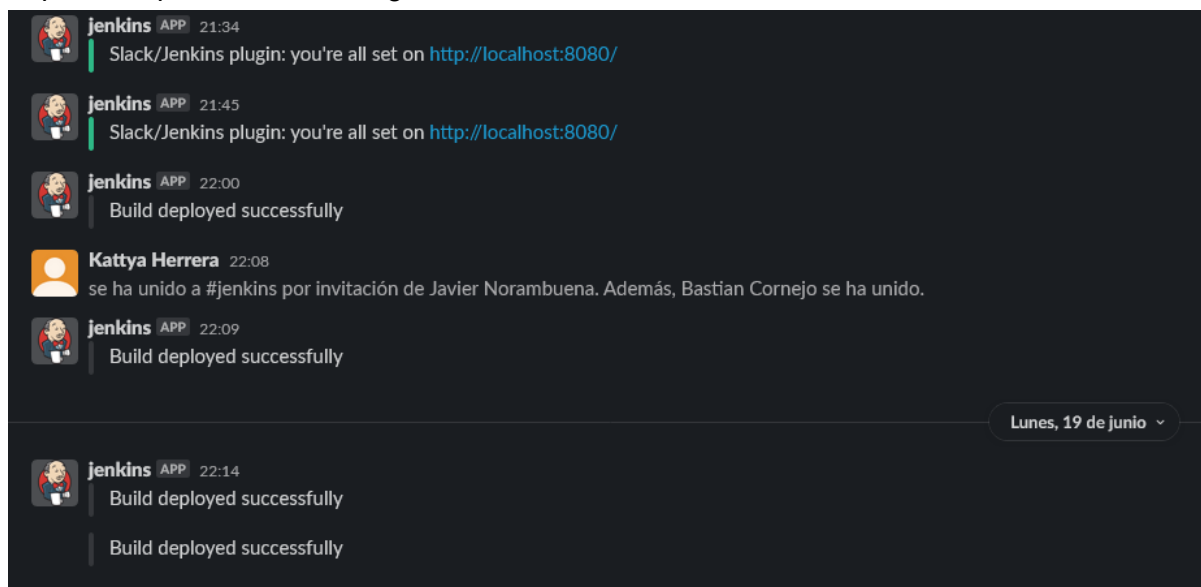
Este es un problema bloqueante ya que hace que el pipeline siempre falle y por lo tanto no se puedan generar nuevas entregas de versiones de la aplicación, es decir, que se actualice el entorno de producción.

Anexo de pruebas de integración

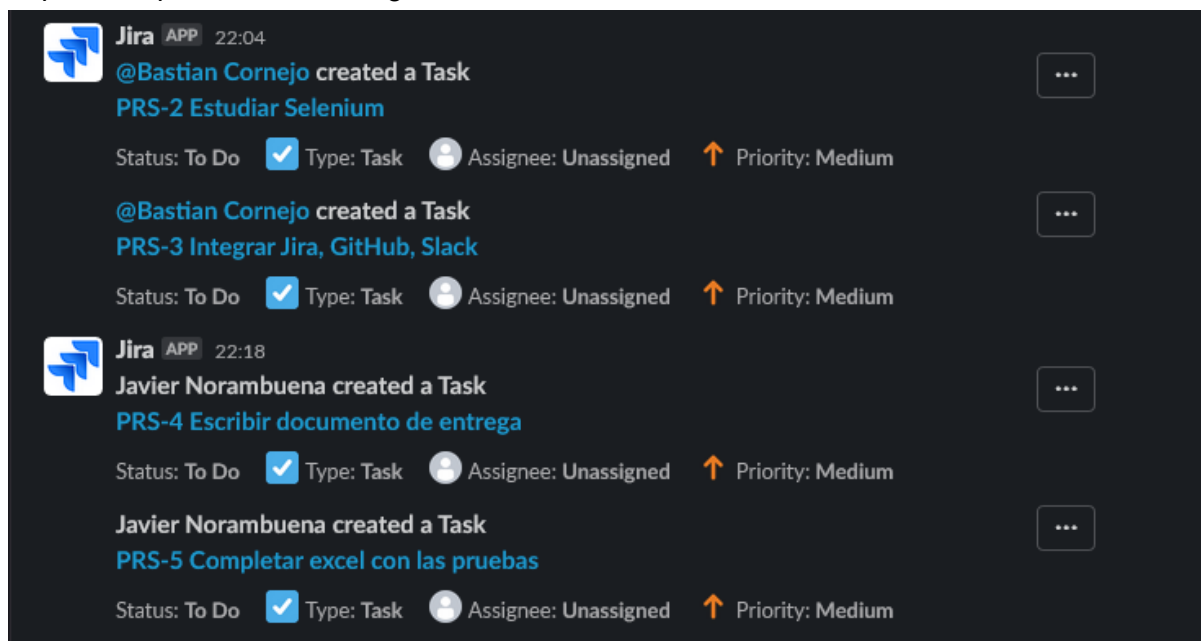
A continuación se muestran las capturas de pantalla de las tecnologías para la gestión del proyecto que fueron integradas al servidor de Slack.

Slack

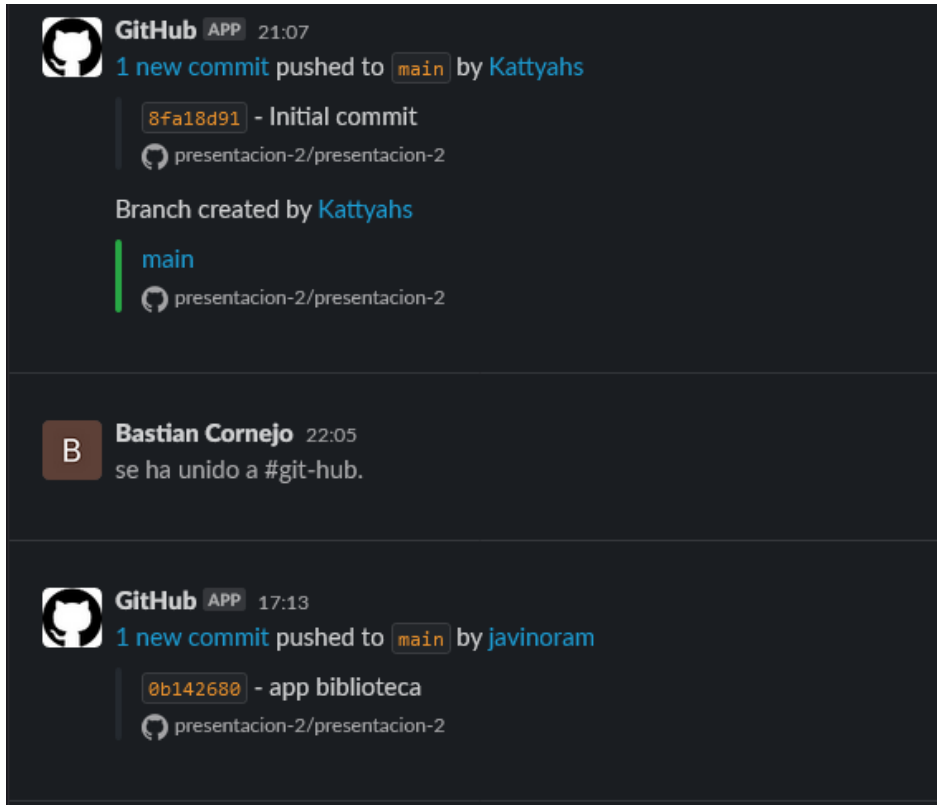
Captura de pantalla de la integración de Jenkins con el servidor de Slack.



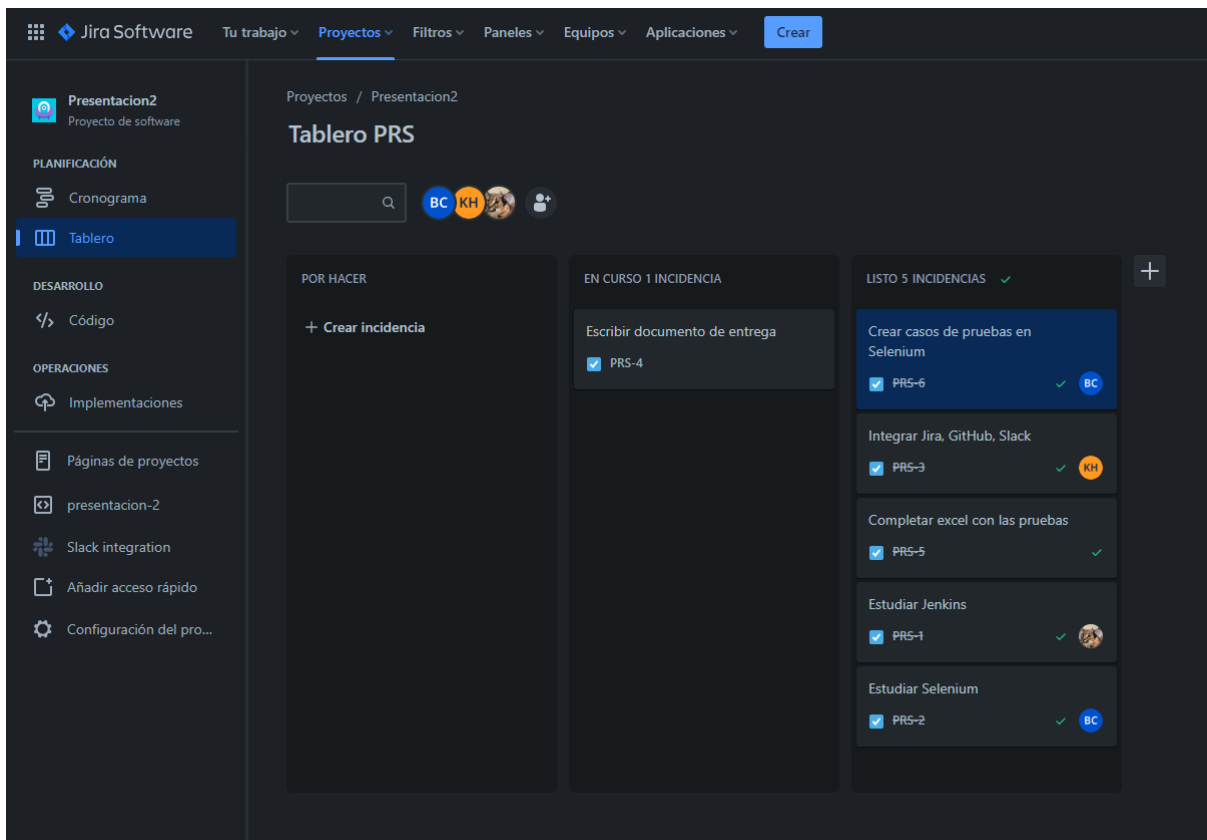
Captura de pantalla de la integración del tablero de Jira con el servidor de Slack.



Captura de pantalla de la integración de Github con el servidor de Slack.



Jira:



Anexo resultado pruebas Selenium report.html

Report generated on 30-Jun-2023 at 22:19:18 by [pytest-html](#) v3.2.0

Summary

14 tests ran in 207.67 seconds.

(Un)check the boxes to filter the results.

☒ 14 passed, ☐ 0 skipped, ☒ 0 failed, ☒ 0 errors, ☒ 0 expected failures, ☒ 0 unexpected passes

Results

[Show all details](#) / [Hide all details](#)

▲ Result	▼ Test	▼ Duration	▼ Links
Passed (show details)	test_TC_0001.py::test_TC_0001	15.69	
Passed (show details)	test_TC_0002.py::test_TC_0002	14.87	
Passed (show details)	test_TC_0003.py::test_TC_0003	14.89	
Passed (show details)	test_TC_0004.py::test_TC_0004	18.21	
Passed (show details)	test_TC_0005.py::test_TC_0005	16.39	
Passed (show details)	test_TC_0006.py::test_TC_0006	13.89	
Passed (show details)	test_TC_0007.py::test_TC_0007	15.67	
Passed (show details)	test_TC_0008.py::test_TC_0008	10.43	
Passed (show details)	test_TC_0009.py::test_TC_0009	13.44	
Passed (show details)	test_TC_0010.py::test_TC_0010	10.50	
Passed (show details)	test_TC_0011.py::test_TC_0011	10.25	
Passed (show details)	test_TC_0012.py::test_TC_0012	11.41	
Passed (show details)	test_TC_0013.py::test_TC_0013	9.44	
Passed (show details)	test_TC_0014.py::test_TC_0014	12.24	